

Java Streams for Beginners (Tier 3)

A continuation of the Tier 1 and Tier 2 guides. By now you have a strong base. This guide covers the next set of stream concepts that come up often in interviews and real code, but are a bit more advanced.

If you have not gone through Tier 1 and Tier 2 yet, please start there first. This guide assumes you know `filter`, `map`, `collect`, `groupingBy`, `partitioningBy`, `reduce`, and the basic terminal operations.

What's New in Tier 3?

In Tier 3, we learn how to:

- Flatten nested data (lists inside lists)
- Create streams from scratch in different ways
- Combine two streams together
- Take or drop items based on a condition
- Debug a stream pipeline
- Work with `Optional` and streams together
- Convert a stream to an array
- Understand how streams actually run (lazy evaluation)
- Use parallel streams safely
- Use a few smart collectors

Let us go through them step by step.

1. `flatMap()` : Flatten Nested Data

If you have a list of lists, or a list of items where each item has more items inside, `flatMap` is what you need.

The simple way to think about it:

- `map` takes one item and gives you one item back.
- `flatMap` takes one item and gives you a stream of items back, then flattens everything into a single stream.

Basic example: list of lists

```
java
```

```
List<List<Integer>> nested = List.of(
    List.of(1, 2, 3),
    List.of(4, 5),
    List.of(6, 7, 8)
);

List<Integer> flat = nested.stream()
    .flatMap(list -> list.stream())
    .collect(Collectors.toList());
// [1, 2, 3, 4, 5, 6, 7, 8]
```

You can also write it with a method reference:

```
java

List<Integer> flat = nested.stream()
    .flatMap(List::stream)
    .collect(Collectors.toList());
```

Why not just use map?

If you used `map` here, you would get a `Stream<Stream<Integer>>`, not a flat list. `flatMap` does the flattening for you.

Splitting strings into words

This is a classic interview question. Given a list of sentences, get all the words.

```
java

List<String> sentences = List.of(
    "hello world",
    "java is fun",
    "streams are powerful"
);

List<String> words = sentences.stream()
    .flatMap(sentence -> Arrays.stream(sentence.split(" ")))
    .collect(Collectors.toList());
// [hello, world, java, is, fun, streams, are, powerful]
```

Get all unique words

```
java
```

```
List<String> uniqueWords = sentences.stream()
    .flatMap(s -> Arrays.stream(s.split(" ")))
    .distinct()
    .sorted()
    .collect(Collectors.toList());
```

flatMap with custom objects

A common case: a list of orders, each with a list of items.

```
java

record Item(String name, double price) {}
record Order(String customer, List<Item> items) {}

List<Order> orders = List.of(
    new Order("Alice", List.of(
        new Item("Book", 100),
        new Item("Pen", 20)
    )),
    new Order("Bob", List.of(
        new Item("Bag", 500)
    ))
);

// Get all items from all orders
List<Item> allItems = orders.stream()
    .flatMap(order -> order.items().stream())
    .collect(Collectors.toList());

// Get total price across all orders
double total = orders.stream()
    .flatMap(order -> order.items().stream())
    .mapToDouble(Item::price)
    .sum();
// 620.0
```

When to use flatMap

If you find yourself with a `Stream<List<X>>` or `Stream<Stream<X>>`, you wanted `flatMap`.

A simple rule: if you say "I want all the X from all the Y", you need `flatMap`.

2. Creating Streams from Scratch

So far we have created streams from a `List` or `Set` using `.stream()`. There are other ways too.

Stream.of() : From specific values

```
java

Stream<Integer> s = Stream.of(1, 2, 3, 4, 5);

Stream<String> names = Stream.of("Alice", "Bob", "Carol");
```

Useful for quick tests or when you have a few known values.

Stream.empty() : An empty stream

```
java

Stream<String> empty = Stream.empty();

long count = empty.count();    // 0
```

Sometimes you need to return an empty stream from a method. This is the way.

Arrays.stream() : From an array

```
java

int[] nums = {1, 2, 3, 4, 5};

int total = Arrays.stream(nums).sum();
// 15

String[] names = {"Alice", "Bob"};
Arrays.stream(names).forEach(System.out::println);
```

For primitive arrays like `int[]`, this gives you an `IntStream` directly.

Stream.generate() : Make an infinite stream

`Stream.generate` keeps producing values forever. You must use `limit` to stop it.

```
java
```



```
// 5 random numbers
Stream.generate(Math::random)
    .limit(5)
    .forEach(System.out::println);

// 3 copies of "hello"
List<String> hellos = Stream.generate(() -> "hello")
    .limit(3)
    .collect(Collectors.toList());
// [hello, hello, hello]
```

If you forget `limit`, your program runs forever. Be careful.

Stream.iterate(): Make a sequence

`Stream.iterate` builds a stream where each value comes from the previous one.

```
java

// Powers of 2
List<Integer> powers = Stream.iterate(1, n -> n * 2)
    .limit(6)
    .collect(Collectors.toList());
// [1, 2, 4, 8, 16, 32]

// Even numbers
List<Integer> evens = Stream.iterate(0, n -> n + 2)
    .limit(5)
    .collect(Collectors.toList());
// [0, 2, 4, 6, 8]
```

Stream.iterate with a stopping rule (Java 9+)

The newer version takes a condition. No need for `limit`.

```
java

List<Integer> upTo100 = Stream.iterate(1, n -> n < 100, n -> n * 2)
    .collect(Collectors.toList());
// [1, 2, 4, 8, 16, 32, 64]
```

The three arguments are: start value, "keep going" condition, and how to make the next value.

IntStream.range and rangeClosed

For a sequence of integers, these are simpler.

```
java
```

```
// 0 to 9 (10 not included)
IntStream.range(0, 10).forEach(System.out::println);

// 1 to 10 (10 included)
IntStream.rangeClosed(1, 10).forEach(System.out::println);
```

These are great when you just need a counter or want to loop a fixed number of times.

From a Map

A `Map` is not a stream itself, but you can stream its entries, keys, or values.

```
java

Map<String, Integer> ages = Map.of(
    "Alice", 25,
    "Bob", 30,
    "Carol", 28
);

// Stream of keys
List<String> names = ages.keySet().stream()
    .sorted()
    .collect(Collectors.toList());

// Stream of values
int totalAge = ages.values().stream()
    .mapToInt(Integer::intValue)
    .sum();

// Stream of entries (most flexible)
ages.entrySet().stream()
    .filter(e -> e.getValue() > 25)
    .forEach(e -> System.out.println(e.getKey() + " is " + e.getValue()));
```

3. Stream.concat(): Combine Two Streams

When you have two streams and want to merge them.

```
java
```

```
Stream<Integer> first = Stream.of(1, 2, 3);
Stream<Integer> second = Stream.of(4, 5, 6);

List<Integer> combined = Stream.concat(first, second)
    .collect(Collectors.toList());
// [1, 2, 3, 4, 5, 6]
```

Combining lists

```
java

List<String> a = List.of("apple", "banana");
List<String> b = List.of("cherry", "date");

List<String> all = Stream.concat(a.stream(), b.stream())
    .collect(Collectors.toList());
// [apple, banana, cherry, date]
```

Combining more than two

`Stream.concat` only takes two streams. For more, you can nest them or use `flatMap`:

```
java

List<String> a = List.of("a", "b");
List<String> b = List.of("c", "d");
List<String> c = List.of("e", "f");

List<String> all = Stream.of(a, b, c)
    .flatMap(List::stream)
    .collect(Collectors.toList());
// [a, b, c, d, e, f]
```

This is cleaner than nesting `Stream.concat` calls.

4. takeWhile() and dropWhile() (Java 9+)

These look like `filter` but work very differently. They check items in order and stop or start based on the first one that fails the test.

takeWhile: Take items until the rule fails

```
java
```

```
List<Integer> nums = List.of(1, 2, 3, 4, 1, 2);

List<Integer> result = nums.stream()
    .takeWhile(n -> n < 3)
    .collect(Collectors.toList());
// [1, 2]
```

It takes items as long as they match. As soon as it sees **3** (which fails the rule), it stops. Even though **1** and **2** come later, they are not included.

dropWhile : Drop items until the rule fails

```
java

List<Integer> result = nums.stream()
    .dropWhile(n -> n < 3)
    .collect(Collectors.toList());
// [3, 4, 1, 2]
```

It throws away items as long as they match. As soon as it sees **3**, it stops dropping. The rest of the stream comes through, including the **1** and **2** at the end.

Compare with filter

```
java

List<Integer> nums = List.of(1, 2, 3, 4, 1, 2);

// filter checks every item
nums.stream().filter(n -> n < 3).collect(Collectors.toList());
// [1, 2, 1, 2]

// takeWhile stops at the first failure
nums.stream().takeWhile(n -> n < 3).collect(Collectors.toList());
// [1, 2]
```

When are these useful?

These shine on sorted data. For example, if you have sorted ages and want everyone under 30:

```
java
```

```
List<Integer> sortedAges = List.of(20, 22, 25, 28, 35, 40);

List<Integer> youngOnes = sortedAges.stream()
    .takeWhile(age -> age < 30)
    .collect(Collectors.toList());
// [20, 22, 25, 28]
```

This is faster than `filter` because it stops as soon as it sees `35`. With unsorted data, use `filter` instead.

5. peek(): Look at the Stream Without Changing It

`peek` is like `forEach`, but it is an intermediate operation. It lets you see what is going through the stream without ending it.

Use it for debugging

```
java

List<Integer> result = List.of(1, 2, 3, 4, 5).stream()
    .peek(n -> System.out.println("input: " + n))
    .filter(n -> n % 2 == 0)
    .peek(n -> System.out.println("after filter: " + n))
    .map(n -> n * 10)
    .peek(n -> System.out.println("after map: " + n))
    .collect(Collectors.toList());
```

This prints each value at every step. Very useful when you do not understand why your stream is giving wrong results.

Important: do not use peek for real work

`peek` is only meant for debugging or logging. Do not use it to change items or save them somewhere.

```
java

// BAD: using peek to add to a list
List<Integer> result = new ArrayList<>();
nums.stream()
    .peek(result::add)    // do not do this
    .count();
```

There are two reasons:

1. The Java compiler can sometimes skip `peek` if it thinks nothing else needs it.

2. It is unclear and confusing to read.

If you want to do something to each item, use `forEach` (terminal) or change items with `map`.

6. Optional with Streams

You will often have a `List<Optional<T>>` or want to handle missing values cleanly. Streams help here.

Filter out empty Optionals (Java 9+)

```
java

List<Optional<String>> opts = List.of(
    Optional.of("Alice"),
    Optional.empty(),
    Optional.of("Bob"),
    Optional.empty()
);

List<String> names = opts.stream()
    .flatMap(Optional::stream)
    .collect(Collectors.toList());
// [Alice, Bob]
```

`Optional::stream` returns a stream of one item if the Optional has a value, or an empty stream if it does not. `flatMap` then flattens everything into a clean list.

Older way (Java 8)

If you are stuck on Java 8:

```
java

List<String> names = opts.stream()
    .filter(Optional::isPresent)
    .map(Optional::get)
    .collect(Collectors.toList());
```

This works but the newer `flatMap(Optional::stream)` is cleaner.

Stream of values that might be missing

Imagine you have a list of IDs and a function that may or may not find a record.

```
java
```

```
record User(int id, String name) {}

List<Integer> ids = List.of(1, 2, 3, 4);

Optional<User> findUser(int id) {
    // returns Optional.of(user) or Optional.empty()
    return Optional.empty();    // dummy
}

List<User> users = ids.stream()
    .map(id -> findUser(id))
    .flatMap(Optional::stream)
    .collect(Collectors.toList());
```

This gives you only the users that were actually found.

7. toArray(): Convert a Stream to an Array

Sometimes you need an array instead of a list.

Stream of objects to Object[]

```
java

Object[] arr = Stream.of("a", "b", "c").toArray();
```

This gives you an `Object[]`, which is rarely what you want.

Stream of objects to a typed array

```
java

String[] arr = Stream.of("a", "b", "c")
    .toArray(String[]::new);
```

The `String[]::new` part tells Java to create a `String[]` of the right size.

Primitive streams to int[], long[], double[]

```
java
```

```
int[] arr = IntStream.range(1, 6).toArray();
// [1, 2, 3, 4, 5]

int[] doubled = List.of(1, 2, 3).stream()
    .mapToInt(Integer::intValue)
    .map(n -> n * 2)
    .toArray();
// [2, 4, 6]
```

For primitive streams, `toArray()` returns the right primitive array directly.

8. Lazy Evaluation : How Streams Actually Work

This is more of a concept than a method, but it is asked in interviews and helps you understand streams better.

Streams are lazy

Intermediate operations like `filter` and `map` do not run when you call them. They only run when a terminal operation is called.

```
java

List<Integer> nums = List.of(1, 2, 3, 4, 5);

Stream<Integer> s = nums.stream()
    .filter(n -> {
        System.out.println("filter: " + n);
        return n > 2;
    })
    .map(n -> {
        System.out.println("map: " + n);
        return n * 10;
    });

System.out.println("Stream is built but nothing has run yet");

List<Integer> result = s.collect(Collectors.toList());
```

Output:


```
Stream is built but nothing has run yet
filter: 1
filter: 2
filter: 3
map: 3
filter: 4
map: 4
filter: 5
map: 5
```

Notice three things:

1. Nothing prints before `collect` is called.
2. Items go through the pipeline one at a time, not in batches. Item 3 goes through filter, then through map, then 4 starts.
3. Items that fail the filter never reach the map.

Why this matters: short-circuit operations

Some operations stop early when they have what they need. This saves a lot of work on big streams.

```
java
List<Integer> nums = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

Optional<Integer> first = nums.stream()
    .filter(n -> {
        System.out.println("checking: " + n);
        return n > 3;
    })
    .findFirst();
```

Output:

```
checking: 1
checking: 2
checking: 3
checking: 4
```

Once it finds `4`, it stops. It never checks `5` through `10`.

This is why streams can be efficient even on huge data sets. They only do what they need to do.

Operations that can short-circuit

- `findFirst`, `findAny`
- `anyMatch`, `allMatch`, `noneMatch`
- `limit`

Operations that cannot

- `sorted` (needs all items first)
- `count` (needs to see all items)
- `collect` (usually needs all items)

A trap: infinite streams need short-circuit operations

```
java
Stream.iterate(1, n -> n + 1)
    .filter(n -> n > 100)
    .findFirst();    // works, finds 101
```

But this would run forever:

```
java
Stream.iterate(1, n -> n + 1)
    .filter(n -> n > 100)
    .count();    // never finishes!
```

Always pair infinite streams with short-circuit operations like `limit`, `findFirst`, or `anyMatch`.

9. Parallel Streams : Use With Care

A parallel stream splits the work across multiple CPU cores. It can be faster for big data, but it has many traps.

How to make one

```
java
```

```
List<Integer> nums = IntStream.rangeClosed(1, 1000000)
    .boxed()
    .collect(Collectors.toList());

int total = nums.parallelStream()
    .mapToInt(Integer::intValue)
    .sum();
```

Just use `parallelStream()` instead of `stream()`. Or call `.parallel()` on an existing stream.

When parallel actually helps

- You have a lot of data (think hundreds of thousands of items)
- The work per item is heavy (math, parsing, etc.)
- The operations are independent (no shared state)
- Order does not matter

When parallel hurts

- Small lists. The setup cost is more than the work itself.
- I/O work like network calls or database reads. Threads sit idle waiting.
- Shared mutable state. Two threads writing to the same list is a bug waiting to happen.
- Order-sensitive work. `findFirst` and `limit` lose most of their speed boost.

A common mistake

```
java

List<Integer> result = new ArrayList<>();

nums.parallelStream()
    .filter(n -> n > 100)
    .forEach(result::add);    // race condition!
```

`ArrayList` is not thread-safe. Two threads adding at the same time can crash the program or lose data. Use `collect` instead:

```
java

List<Integer> result = nums.parallelStream()
    .filter(n -> n > 100)
    .collect(Collectors.toList());
```

`collect` knows how to gather results safely from multiple threads.

A simple rule

Start with regular `stream()`. Only switch to `parallelStream()` if you have measured a real performance problem and you know the data is big and the work is independent. Do not sprinkle `parallelStream` everywhere "just in case".

10. A Few Smart Collectors

These collectors are nice to know but rarely needed every day.

Collectors.toUnmodifiableList() (Java 10+)

Returns a list you cannot change.

```
java

List<Integer> immutable = nums.stream()
    .filter(n -> n > 2)
    .collect(Collectors.toUnmodifiableList());

immutable.add(100);    // crash: UnsupportedOperationException
```

In Java 16+, `.toList()` does the same thing. Both are good for return values from methods, so callers cannot modify your data.

Collectors.teeing() (Java 12+)

Run two collectors at once and combine their results.

```
java

record Stats(double avg, long count) {}

List<Integer> nums = List.of(10, 20, 30, 40, 50);

Stats result = nums.stream()
    .collect(Collectors.teeing(
        Collectors.averagingInt(Integer::intValue),
        Collectors.counting(),
        Stats::new
    ));

System.out.println(result);
// Stats[avg=30.0, count=5]
```

The three arguments are: first collector, second collector, and how to combine their results.

This is useful when you want two stats at once without going through the data twice.

Putting It All Together

Real examples that mix these Tier 3 concepts.

Example 1: Get all unique words from a list of sentences, sorted by length

```
java

List<String> sentences = List.of(
    "java is fun",
    "streams are powerful",
    "code is poetry"
);

List<String> uniqueWords = sentences.stream()
    .flatMap(s -> Arrays.stream(s.split(" ")))
    .distinct()
    .sorted(Comparator.comparingInt(String::length))
    .collect(Collectors.toList());
```

Example 2: Get all items from all orders that cost more than 100

```
java

record Item(String name, double price) {}
record Order(String customer, List<Item> items) {}

List<Item> expensive = orders.stream()
    .flatMap(order -> order.items().stream())
    .filter(item -> item.price() > 100)
    .collect(Collectors.toList());
```

Example 3: Build a list of the first 10 squares

```
java

List<Integer> squares = IntStream.rangeClosed(1, 10)
    .map(n -> n * n)
    .boxed()
    .collect(Collectors.toList());
// [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

`boxed()` turns an `IntStream` into a `Stream<Integer>` so you can collect it into a `List`.

Example 4: Filter out missing optionals from a list of search results

```
java

List<Optional<User>> results = ids.stream()
    .map(this::findUser)
    .collect(Collectors.toList());

List<User> found = results.stream()
    .flatMap(Optional::stream)
    .collect(Collectors.toList());
```

Example 5: Take items from a sorted list until a condition fails

```
java

List<Integer> sortedScores = List.of(95, 90, 85, 80, 75, 70, 65);

List<Integer> topScores = sortedScores.stream()
    .takeWhile(score -> score >= 80)
    .collect(Collectors.toList());
// [95, 90, 85, 80]
```

Example 6: Combine two lists and remove duplicates

```
java

List<String> teamA = List.of("Alice", "Bob", "Carol");
List<String> teamB = List.of("Bob", "Dan", "Eve");

List<String> allMembers = Stream.concat(teamA.stream(), teamB.stream())
    .distinct()
    .sorted()
    .collect(Collectors.toList());
// [Alice, Bob, Carol, Dan, Eve]
```

Example 7: Calculate average and count in one pass

```
java
```

```
record Stats(double avg, long count) {}

Stats stats = nums.stream()
    .collect(Collectors.teeing(
        Collectors.averagingInt(Integer::intValue),
        Collectors.counting(),
        Stats::new
    ));
```

Common Mistakes in Tier 3

Mistake 1: Forgetting that infinite streams need limits

```
java

Stream.generate(Math::random).forEach(System.out::println);
// Runs forever!
```

Always pair `generate` and basic `iterate` with `limit` or a short-circuit operation.

Mistake 2: Using peek for real work

```
java

nums.stream()
    .peek(System.out::println) // OK for debug
    .peek(saveToDatabase)      // BAD, do not do this
    .count();
```

Use `forEach` if you really need to do something with each item.

Mistake 3: Using map when you need flatMap

```
java

List<List<Integer>> nested = List.of(List.of(1, 2), List.of(3, 4));

// Wrong: gives Stream<Stream<Integer>>
nested.stream().map(List::stream).collect(Collectors.toList());

// Right
nested.stream().flatMap(List::stream).collect(Collectors.toList());
```

If you end up with a stream of streams, you wanted `flatMap`.

Mistake 4: Trying to sort an infinite stream

```
java
Stream.iterate(1, n -> n + 1)
    .sorted()
    .limit(10);    // hangs forever
```

`sorted` needs to see all items first. On an infinite stream, that never happens.

Mistake 5: Using `parallelStream` without thinking

```
java
List<Integer> tiny = List.of(1, 2, 3);
tiny.parallelStream().map(n -> n * 2).collect(Collectors.toList());
```

For small lists, parallel is slower because of the setup cost. Default to `stream()`.

Mistake 6: Modifying shared state in parallel streams

```
java
Map<String, Integer> counts = new HashMap<>();

words.parallelStream().forEach(word -> {
    counts.merge(word, 1, Integer::sum);    // bad, not thread-safe
});
```

Use `collect` with `groupingBy` and `counting` instead. It is safe and clean.

```
java
Map<String, Long> counts = words.parallelStream()
    .collect(Collectors.groupingBy(w -> w, Collectors.counting()));
```

Quick Cheat Sheet

You want to...	Use
Flatten a list of lists	<code>flatMap</code>
Flatten by splitting strings	<code>flatMap(s -> Arrays.stream(s.split(...)))</code>
Make a stream from values	<code>Stream.of(...)</code>

You want to...	Use
Make an empty stream	<code>Stream.empty()</code>
Make an infinite stream	<code>Stream.generate(...)</code> or <code>Stream.iterate(...)</code>
Get a number range	<code>IntStream.range(start, end)</code> or <code>rangeClosed</code>
Combine two streams	<code>Stream.concat(a, b)</code>
Combine many streams	<code>Stream.of(a, b, c).flatMap(...)</code>
Take items until a rule fails	<code>takeWhile</code>
Drop items until a rule fails	<code>dropWhile</code>
Print items as they pass through	<code>peek</code> (debug only)
Stream a Map's keys, values, or entries	<code>map.keySet().stream()</code> , etc.
Drop empty Optionals from a stream	<code>flatMap(Optional::stream)</code>
Convert to array	<code>toArray()</code> or <code>toArray(Type[]::new)</code>
Convert IntStream to Stream of Integer	<code>boxed()</code>
Get unmodifiable list	<code>Collectors.toUnmodifiableList()</code> or <code>.toList()</code>
Two collectors at once	<code>Collectors.teeing(...)</code>
Run on multiple cores	<code>parallelStream()</code> (use carefully)

Final Tips

1. Master `flatMap` first. It is the most useful Tier 3 concept and shows up in lots of interview questions.
2. Understand lazy evaluation. It explains why streams are efficient and why some pipelines surprise you.
3. Be careful with infinite streams. Always pair them with `limit` or a short-circuit operation.
4. Do not reach for `parallelStream` unless you have a real performance problem and you understand the trade-offs.
5. `peek` is for debugging only. For real work, use `forEach` or `map`.
6. Think about whether you really need each Tier 3 tool. Most code can be written cleanly with just Tier 1 and Tier 2 operations. Use Tier 3 when it actually makes things simpler.

You now have a solid grasp of nearly every Stream operation you will see in interviews and day-to-day work. Practice combining these in small pipelines until the patterns feel natural.

If you want to keep going, the next step is to look at custom collectors, deeper parallel stream tuning, and integrating streams with other Java features like records, sealed types, and switch expressions.